

Pretraining Objective and Language Modeling

pig7selene

September 5, 2026

Abstract

This chapter formalizes the Causal Language Modeling pretraining objective from the vocabulary logits produced by a decoder-only Transformer. It derives autoregressive sequence factorization, next-token prediction, and the relationship between token-level negative log-likelihood and cross-entropy. It distinguishes vocabulary size, sequence length, logits, probability distributions, targets, and loss aggregation, then explains why teacher forcing and causal masking allow eligible token losses to be evaluated in parallel even though generation proceeds one token at a time.

1 Introduction

The first four chapters established how a token sequence enters a decoder-only Transformer, how causal Self-Attention and Feed-Forward Networks update its residual stream, and how final hidden states produce vocabulary logits. Pretraining assigns a statistical task to that computation graph: to learn a probability distribution over token sequences. For a tokenized sequence $x = (x_1, \dots, x_T)$, the model does not choose an entire text from an unstructured space at once. Instead, at each position it assigns probabilities to possible next tokens conditioned on the prefix that has already appeared.

This objective is central to modern autoregressive language models. It turns unlabeled text into a supervised signal: the same sequence supplies both context and target. Neural probabilistic language modeling seeks to model the joint probability of token sequences [1]; GPT-style models scale this principle into the pretraining objective of large decoder-only Transformers [2].

This chapter retains the token notation of Chapter 1. Let $\mathcal{V}$ denote the vocabulary and let $V = |\mathcal{V}|$ be its size. Let T denote the token length of a particular sequence; it is not a property of the vocabulary. For a batch, let B be the number of sequences. The language-model head introduced in Chapter 2 produces a logit vector of length V for every batch item and position. The remaining question is how these vectors become conditional probabilities and a training loss.

2 Autoregressive Language Modeling

Autoregressive Language Modeling represents a sequence from left to right. At prediction position t , the model may use only the token prefix strictly to its left, denoted by $x_{\{<t\}} = (x_1, \dots, x_{t-1})$. With parameters θ , its conditional distribution over any vocabulary token v is written as

$$p_{\theta}(v \mid x_{\{<t\}}), \quad v \in \mathcal{V}. \quad (1)$$

This quantity is not a single score for a known target; it is a complete distribution over the vocabulary. It is nonnegative and normalized: $p_{\theta}(v \mid x_{\{<t\}}) \geq 0$ and $\sum_{v \in \mathcal{V}} p_{\theta}(v \mid x_{\{<t\}}) = 1$. During generation, this distribution provides candidates for the next token. During pretraining, the corpus supplies the token that actually follows the prefix, so the model can evaluate the probability it assigned to that target. For variable-length text, sequence termination must also be represented by the probability model. Implementations usually include an end-of-sequence token and train the model to predict it at an appropriate position. To avoid adding boundary notation to each derivation, the sequence $x_1, \dots, x_T$ below is understood to include the required boundary convention.

3 Sequence Probability Factorization

The probability chain rule factorizes any joint distribution. An autoregressive model uses parameterized conditional distributions to instantiate that factorization:

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{\{<t\}}). \quad (2)$$

The factorization does not declare tokens independent. On the contrary, the term at position t may depend on the full prefix. It specifies the direction of dependence: the model may summarize prior tokens in its hidden state, but it may not read x_t itself or any future token when predicting x_t . The causal Attention Mask described in Chapter 3 is the Transformer mechanism that enforces this information constraint.

The product form expresses a probability, but it is inconvenient for numerical optimization. Taking a logarithm turns the sequence log-probability into a sum of conditional log-probabilities:

$$\log p_{\theta}(x_1, \dots, x_T) = \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{\{<t\}}). \quad (3)$$

Consequently, maximizing the likelihood of whole sequences is equivalent to maximizing the conditional log-probability of every observed next token. This equivalence is why a local next-token training signal defines a sequence-level probability model.

4 Next-Token Prediction

In training code, next-token prediction is usually implemented by a shift. If a beginning-of-sequence token x_0 is introduced, the model input is $(x_0, x_1, \dots, x_{\{T-1\}})$, and the output at position $t-1$ predicts the target x_t . Implementations without an explicit x_0 commonly shift the inputs and labels of a packed sequence by one position: the logits at position t align with the token at position $t+1$. Both conventions have the same semantics: the prediction of target x_t is conditioned only on $x_{\{<t\}}$. The target token is an integer ID, not a one-hot vector and not a token generated by the model. For batch item b and supervised position t , write the target as $y_{b,t} \in \{0, \dots, V-1\}$. Under the common right-shift convention, $y_{b,t}$ is the next token in the input sequence. An off-by-one error in this alignment changes the training objective itself; it is not a small discrepancy that an optimizer can correct.

5 Logits and Token Probabilities

Chapter 2 denotes the output of the language-model head by $O \in R^{B \times T \times V}$. The vector $O_{b,t,:}$ has length V and is called the logits at position (b, t) . Its entries are unnormalized scores, not probabilities. For fixed (b, t) , softmax defines

$$p_{\theta}(v \mid x_{b,<t}) = \frac{\exp(O_{b,t,v})}{\sum_{j=0}^{V-1} \exp(O_{b,t,j})}. \quad (4)$$

Softmax maps an arbitrary real-valued vector to the vocabulary simplex. Adding a common constant to all logits leaves Equation 4 unchanged, so the absolute zero point of a logit has no probabilistic meaning; only relative scores among vocabulary items determine the distribution. The vocabulary size V determines the width of each logit vector, whereas T determines the number of prediction positions. They are distinct quantities.

Quantity	Symbol or shape	Meaning
Vocabulary size	$V = \mathcal{V} $	Number of possible token IDs and width of every logit vector.
Sequence length	T	Number of token positions in one sequence; it varies with the tokenized text.
Logits	$O_{b,t,:} \in R^V$	Unnormalized scores at one prediction position.
Probability distribution	$p_{\theta}(\cdot \mid x_{b,<t})$	Softmax-normalized categorical distribution over the vocabulary.
Target token	$y_{b,t}$	Observed integer token ID used to select one probability.
Token-level loss	$\ell_{b,t}$	Negative log-probability assigned to one observed target.
Sequence / batch loss	$\mathcal{L}_b, \mathcal{L}_{\text{batch}}$	Aggregations of token losses with an explicit reduction rule.

Table 1: Objects that are often conflated in language-model training. A target is one index; logits and probabilities are length- V vectors; losses reduce these objects to scalars.

6 Cross-Entropy and Negative Log-Likelihood

For one supervised position, define the empirical target distribution $q_{b,t}$ to place all of its probability mass on $y_{b,t}$. The cross-entropy from this one-hot target distribution to the model distribution is

$$\ell_{b,t} = - \sum_{v=0}^{V-1} q_{b,t}(v) \log p_{\theta}(v \mid x_{b,<t}) = - \log p_{\theta}(y_{b,t} \mid x_{b,<t}). \quad (5)$$

The final expression is the token-level negative log-likelihood (NLL). When the target is one-hot, cross-entropy and NLL refer to the same numerical training term. Their conceptual emphases differ: cross-entropy compares two distributions, whereas NLL emphasizes the probability assigned by the model to an observed sample. Standard information theory relates cross-entropy to both optimal code length and likelihood [3].

Expanding the softmax gives an expression directly in terms of logits:

$$\ell_{b,t} = -O_{b,t,y_{b,t}} + \log \sum_{v=0}^{V-1} \exp(O_{b,t,v}). \quad (6)$$

The first term encourages a high logit for the correct token. The second, commonly called the log-sum-exp term, accounts for every competing vocabulary item. The derivative with respect to any logit is

$$\frac{\partial \ell_{b,t}}{\partial O_{b,t,v}} = p_{\theta}(v \mid x_{b,<t}) - \delta_{v,y_{b,t}}, \quad (7)$$

where $\delta_{v,y}$ is one when $v = y$ and zero otherwise. Gradient descent therefore raises the target logit and lowers competing logits in proportion to their predicted probabilities. This local expression also shows that exact full-softmax loss requires the language-model head to produce scores for all V vocabulary items.

Numerical implementations should not first compute $\exp(O_{b,t,v})$ directly. Let $a = \max_v O_{b,t,v}$. The algebraically equivalent identity

$$\log \sum_v \exp(O_{b,t,v}) = a + \log \sum_v \exp(O_{b,t,v} - a) \quad (8)$$

ensures that the arguments to the exponentials are no greater than zero. In practice, frameworks commonly fuse this stable log-sum-exp calculation with the target gather operation rather than materializing a separate probability tensor solely for the loss.

7 Teacher Forcing and Causal Masking

Teacher forcing means that the prefix supplied during training is drawn from the observed corpus rather than from the model’s earlier samples. The term predates the Transformer [4], but the principle applies directly: when scoring x_t , the model receives $x_{\{<t\}}$ from the data and evaluates a token-level loss against the known target. Every eligible token position in a sequence can therefore contribute a learning signal.

Causal masking makes teacher forcing statistically valid for autoregressive modeling. In a decoder-only attention layer, position t may attend only to permitted earlier positions; future tokens are masked before the attention softmax. The mask prevents a representation from encoding the very answer it will be asked to predict. The mask does not itself define the loss: the loss is defined by target alignment and token-level cross-entropy. The original Transformer applies such a mask in decoder self-attention so that a prediction cannot depend on subsequent positions [5].

This causal dependency does not require a sequential training loop. Given a complete input tensor, projections, masked attention scores, FFNs, the language-model head, and token losses at all $B \times T$ positions can be computed with batched tensor operations. The mask imposes the same allowed-prefix rule on every row of this parallel computation. Generation is different because the next input token is not yet known: the model produces a distribution for the current prefix, selects or samples a token, appends it, and only then can compute the next distribution. Parallel training and sequential generation are thus two execution regimes of the same conditional model, not competing definitions of autoregression.

8 Loss Averaging Across Tokens and Batches

The NLL of a complete sequence is the sum of its token-level losses:

$$\mathcal{L}_b = -\log p_{\theta}(x_{b,1}, \dots, x_{b,T}) = \sum_{t=1}^T \ell_{b,t}. \quad (9)$$

Training usually reports and differentiates a mean loss rather than a raw sum. Let $m_{b,t} \in \{0, 1\}$ indicate whether position (b, t) has a valid supervised target, and let $N = \sum_{b=1}^B \sum_{t=1}^T m_{b,t}$. The token-averaged batch loss is then

$$\mathcal{L}_{\text{batch}} = \frac{1}{N} \sum_{b=1}^B \sum_{t=1}^T m_{b,t} \ell_{b,t}. \quad (10)$$

The loss mask m differs from the attention mask. A causal attention mask constrains information flow; a loss mask specifies which output positions contribute to the scalar objective. Padding positions, positions without a shifted target, and intentionally excluded labels should have $m_{b,t} = 0$. Dividing by N gives each valid token the same weight even when sequences have different effective lengths. By contrast, averaging within each sequence and then averaging sequences gives short and long sequences equal weight. Neither reduction is universally incorrect, but the choice changes the objective and should be recorded with the reported loss.

9 Perplexity

With natural logarithms and the token-averaged NLL $\mathcal{L}_{\text{batch}}$, perplexity is defined as

$$\text{PPL} = \exp(\mathcal{L}_{\text{batch}}). \quad (11)$$

For a single sequence without masked positions, it is the reciprocal geometric mean of the probabilities assigned to its observed tokens. A value near one means that the model assigns high probability to the evaluated token stream; a larger value denotes greater average uncertainty. If cross-entropy is measured in bits with base-two logarithms, the equivalent expression is 2^H .

Perplexity converts average log-loss into an interpretable effective branching factor, but it is not a tokenizer-independent measure of language understanding. As Chapter 1 explains, tokenization

changes both the event space and the number of prediction positions. Token-level PPL values obtained under different vocabularies, normalization policies, or loss masks are not directly comparable. Nor does a lower PPL determine which decoding rule should be used during generation.

10 Training Objective versus Generation

During pretraining, likelihood evaluation asks a non-counterfactual question: given the true prefix from the corpus, how much probability does the model assign to the true next token? Although the complete sequence is available in memory, causal masking ensures that each prediction uses only its permitted prefix. The result is a scalar objective that aggregates many such conditional probabilities. During generation, only the prefix is available. The model produces $p_{\theta}(\cdot \mid x_{\{<t\}})$, and a decoding rule selects or samples a token to extend the prefix. That generated token becomes part of the context for the next step. Maximum-likelihood training specifies how to fit the conditional distribution; it does not prescribe greedy selection, temperature scaling, or any other decoding policy. These inference choices belong to later chapters, but the distinction is essential: token-level training loss evaluates probabilities under data prefixes, whereas generation constructs trajectories under model-produced prefixes.

11 Implementation Contracts

A minimal Causal Language Modeling training step should make four aligned tensors explicit: input token IDs of shape $B \times T$, target IDs of shape $B \times T$, logits of shape $B \times T \times V$, and a loss mask of shape $B \times T$. At every supervised position, the target must be the token immediately following the input prefix represented by the corresponding logit. Wherever the loss mask is one, target IDs must be integer indices in $[0, V - 1]$, not floating-point probabilities.

The loss implementation should consume raw logits, use a stable fused cross-entropy kernel, and reduce only over valid targets. Tests should check the shift explicitly on short hand-written sequences, verify that masked targets contribute zero, and compare a small batch against a direct probability calculation in high precision. A causality test should alter a future input token and confirm that logits at earlier positions do not change. These contracts turn the mathematical objective into a reproducible tensor program.

12 Summary

Causal Language Modeling assigns a probability to a token sequence by factorizing its joint probability into next-token conditionals. A decoder-only Transformer produces a vocabulary-sized logit vector at each position; softmax converts it into a conditional distribution, and the negative log-probability of the observed target gives the token-level cross-entropy. Summing these terms yields the sequence NLL, while a loss-masked token average yields the usual batch loss and its corresponding perplexity. Teacher forcing provides observed prefixes during training, and causal masking prevents those prefixes from containing future answers. Together, they allow position-wise losses across an entire batch to be evaluated in parallel. Autoregressive generation remains sequential because each newly selected token changes the prefix for the next step. The objective is compact, but tensor alignment, masking, reduction conventions, and numerical implementation jointly determine the pretraining problem actually being solved.

References

- [1] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, “A Neural Probabilistic Language Model,” *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003, [Online]. Available: <https://jmlr.org/papers/v3/bengio03a.html>
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” technical report, 2019. [Online]. Available: <https://cdn.openai.com/better-language-models/language-models.pdf>
- [3] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006. doi: 10.1002/047174882X.
- [4] R. J. Williams and D. Zipser, “A Learning Algorithm for Continually Running Fully Recurrent Neural Networks,” *Neural Computation*, vol. 1, no. 2, pp. 270–280, 1989, doi: 10.1162/neco.1989.1.2.270.
- [5] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.